

TESTING

→ Hace evidente defectos. (los que pudo encontrar).
Seguro quedan otros por encontrar.

Hacer testing No significa asegurar calidad.

Calidad → Nivel de cumplimiento a las necesidades y expectativas.

↳ Tampoco puede asegurar que el suu No tenga defectos.

Manifiestos - No manifiestos

↓
No les dicen.

"Era obvio"

• Estudiar los títulos p/ saber identificar qué nos forman.

PUD

- Requerimientos

- Análisis

- Diseño

¿Es el producto correcto? **Validación**

Pruebas unitarias →

- implementación

P. Integración ↗

- **Prueba**

¿Funciona correctamente? **Verificación**

P. de sistema ↗

- Despliegue

P. de aceptación ↗

Error → Queda en la misma fase donde apareció. Es un error de req. que no pasó a las otras fases

Defecto → Error que pasa a otra fase

El testing "llega tarde". Se prueba algo que se construyó antes. Entonces encuentra defectos.

No podemos hacer testing si no tenemos una buena administración de configuración de suu (SCM)

Pruebas estáticas y dinámicas. Lo vemos con caja negra y caja blanca.

Fallo → Mal funcionamiento del sw por un error o defecto.

Hay defectos que no provocan fallos. Ej defectos de diseño de interfaz (colores, errores de ortografía, mensajes de error, etc)

¿Todos los defectos son iguales? No.

El equipo de testing no quiere asignar la severidad.

Severidad

- 1- Bloqueante: No se puede usar el sistema. El sw me las da.
- 2- Crítico: No puedo cumplir mi obj con esta funcionalidad. ↑
- 3- Mayor: No puede cumplir con el obj pero tengo formas alternativas de hacerlo
- 4- Menor: Tengo que resolver pero no me impide el cumplimiento de la funcionalidad.
- 5- Cosmético: Errores de ortografía disposición

Prioridad: Tiene que ver con la urgencia del negocio. La asigna el PO. En los peores casos, el cliente.

1- Urgencia: No puedo esperar nada.

¿Cuánto puede esperar?

2- Alta

3- Media

4- Baja

NIVELES DE PRUEBA

Excepción

1- Pruebas unitarias. Las tiene que hacer el desarrollador. También llamadas primer nivel de testing. Integrados en TDD

2- Pruebas de integración/interfases:

Continuous integration: Se corren las pruebas de integración automáticamente.

3- Pruebas de sistema/de versión: La tiene que hacer alguien independiente de quien construyó. El jefe del eq de desarrollo no debería ser el mismo del eq de testing. Funcionales y no funcionales.

4- Pruebas de aceptación de usuario (UAT) las realiza el usuario con el equipo presente.

Distinto foco de las primeras 2 con las últimas 2

verticales - funcionalidad completa

Ambiente: Configuración del espacio de trabajo con el repositorio y la instalación de todo lo que se necesita para trabajar.

Ambiente de desarrollo / Ambiente de testing

La config. del ambiente de despliegue deberían ser 2

Pre-Producción

Producción

El ambiente de pre-producción es muy caro.

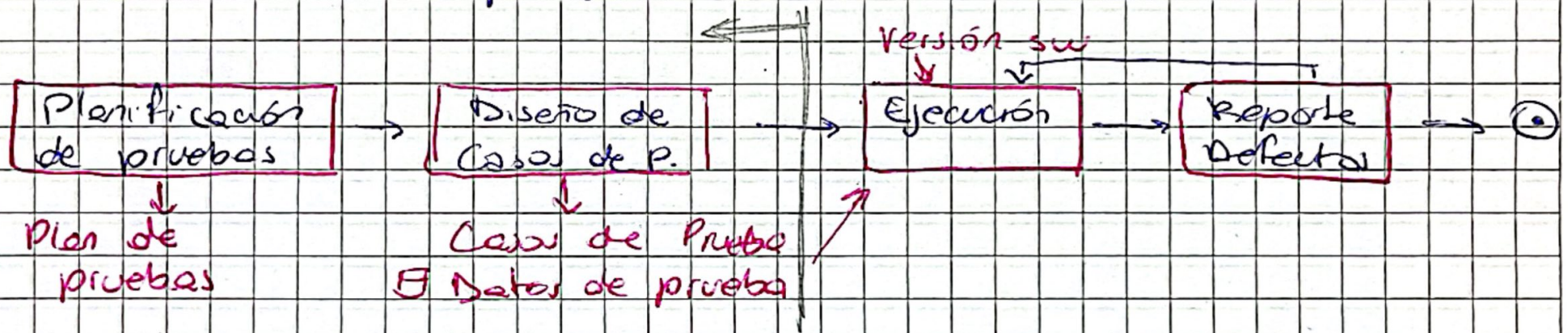
Artefacto más importante del testing. Cards de prueba.

Un defecto que no se puede reproducir, no es defecto.

El testing no deberia ser exhaustivo.

Disminuir la cantidad de casos de prueba sin disminuir la calidad

Vamos a sacar las pruebas desde las user stories.



Sprint 0

Ciclo 0: Encuentro de defectos. Viene la versión nueva

Ciclo 1: No encuentro defectos

Escenario ideal.

2 ciclos.

Regresión Pruebo todo como en el ciclo 0